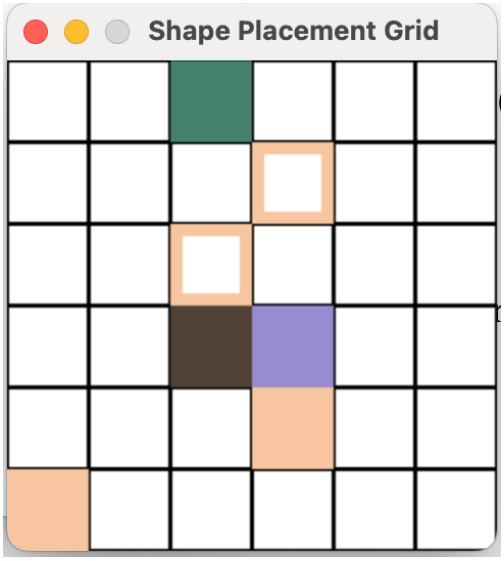
Homework 1: Grid Coloring

Before you start:

Grid Coloring

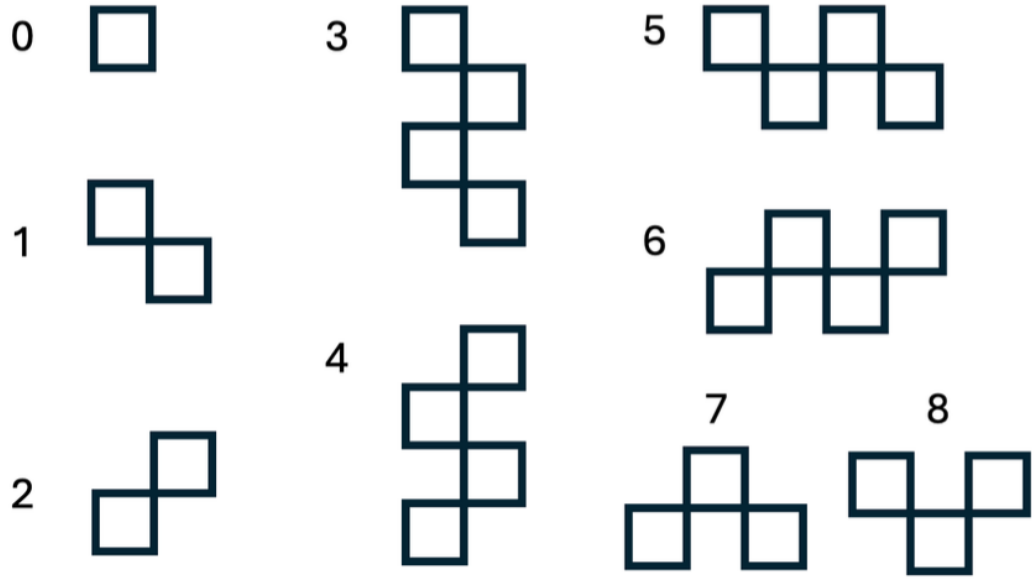


In this assignment, you are given a square grid. A few squares within the grid have been colored. Your agent's goal is to color the remainder of the squares such that adjacent squares do not have the same color. Two squares are considered adjacent if they are orthogonally agent—that is, they touch horizontally or vertically. (This means that two squares *can* have the same color if they only touch diagonally.)

There are four possible colors, which your agent can freely cycle between. Additionally, the agent can color multiple squares at the same time by using a "brush" that covers multiple squares. There are 9 possible brushes, depicted on the next page. All squares covered by the same brush will be painted the same color. Brushes can be freely moved around the grid but cannot be rotated.

You can attempt to complete this task manually by running gridgame.py from a command line terminal. You will need to have the pygame and numpy packages installed.

Brushes



Your Task

Your goal in this assignment is to build an agent that can complete the grid coloring problem for an arbitrary grid. Specifically, your goal is to achieve this coloring using as few shapes and colors as possible–a larger brush may cover multiple cells but counts as one shape. While there are many ways to approach this problem, for this assignment you will be using a local search approach, specifically **first choice hill climbing**. This means that at each point in time, your agent should select a random move and take it if (and only if) it improves the objective function. You may be able to improve performance by using slight variants of this algorithm, such as allowing "sideways" movement, allowing occasional "downhill" moves, or implementing random restarting. If you choose to implement one of these variants, please add a comment to your code explaining it.

I encourage you to think early about how you plan to implement the local search. Having a well-defined state space, action space, and objective function will be crucial to success here!

The Code

Write your solution in the file hw1.py where indicated. (I recommend reading the comments in this file for more information about implementation.) You should **not** edit gridgame.py or otherwise modify any existing code[1]. Whenever your agent wishes to take actions or otherwise interact with the environment, it must be done through the execute() method. This method takes in a single argument, which indicates what action will be taken. A list of arguments is below.

---

[1] The only exception to this is if you cannot easily distinguish between the four colors, you may adjust the colors.

- **export**: returns the current state of the grid, a list of shapes with positions and colors currently placed on the grid, and a Boolean indicating whether the coloring constraints have been satisfied.
- **up/down/left/right**: move the brush in the specified direction by one cell, if possible. (The brush will never move to a position where one of its squares leaves the grid.) The brush starts in the top left corner of the grid when the program is executed.
- **place**: place a shape on the grid, i.e. color the cells covered by the brush in the currently selected brush color, assuming all cells covered by the brush are currently empty.
- **switchshape**: cycle to the next brush shape option.
- **switchcolor**: cycle to the next brush color option.
- **undo**: undo the last placed shape.

Additionally, regardless of what action was taken, execute() will return information about the current environment state. The list of outputs follows.

- **shapePos:** Current location of the brush as a list of size 2 with the x and y coordinates
- **currentShapeIndex:** index of the currently selected brush (using the numbering shown on the diagram in "Brushes")
- **currentColorIndex:** index of the currently selected color (found in the list self.colors)
- **grid:** current grid, as represented by a numpy array. More information about how grids are converted to numpy arrays can be found in hw1.py.
- **placedShapes:** a list of all shapes placed so far
- **done:** a Boolean that is true if (and only if) all cells have been colored, and no adjacent cells share a color

In your code, you should never take actions or update the environment in any way without using execute. However, you may wish to make a copy of the game grid, in order to compute your objective function, check if an action is legal, or otherwise aid in your search. You may also use the other **public** functions of gridgame.py (canPlace, checkGrid, and getAvailableColor) in order to help you implement your search. You should not use the private functions in gridgame.py. (These functions begin with _ and should be clearly marked in the file.)

Grading Criteria

Due to the nature of hill climbing methods, you are not expected to find the optimal coloring for every grid. However, your agent should consistently find a valid coloring, should attempt to minimize the number of colors and shapes used, and should complete the search in an efficient manner. We will evaluate how well your agent succeeds using a mix of autograding and manual grading.

The autograder will run your code on three different grids. The autograder has a time limit of 10 minutes, so any implementations that do not finish in that time will not receive full credit. For each test case that finishes, the autograder will check whether your code produces a valid coloring, as well as count how many shapes and colors it uses. Solutions that do not

produce valid colorings or use an excessive number of shapes and colors will be penalized. I recommend uploading your code early to verify it works with the autograder.

Manual grading will be used to verify that you fit the assignment criteria (such as properly using the execute() function and not modifying any additional code in gridgame), as well as determining how well you implemented the first choice hill climbing to achieve your goal. Assignments that do not use the correct local search method (even if they otherwise pass the autograder) will lose points in manual grading, even if they pass the autograder's tests. In particular, hardcoded solutions, or other solutions written solely to work on the autograder test cases will automatically receive a 0.